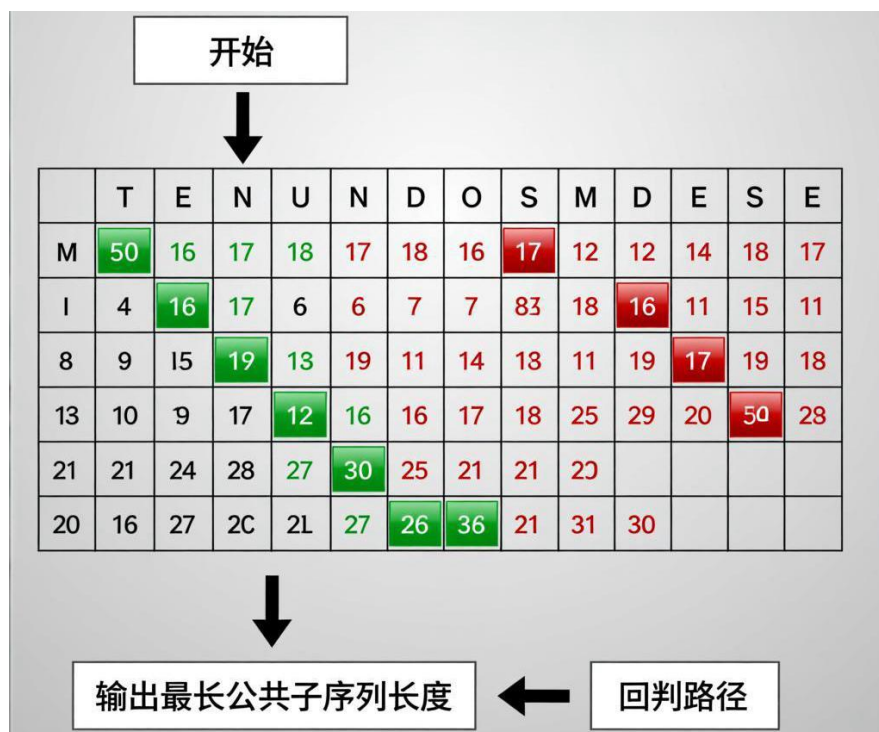


动规解LCS：从递推到最优解



字符串动态规划

字符串动态规划是指采用动态规划算法求解字符串的相关问题。

王晓茹

北京邮电大学



目录

CONTENT

01 | 问题与最优子结构

02 | 递推式与自底向上

03 | DP表推演与路径标记

04 | 最优解重建

05 | LCS解LIS技巧

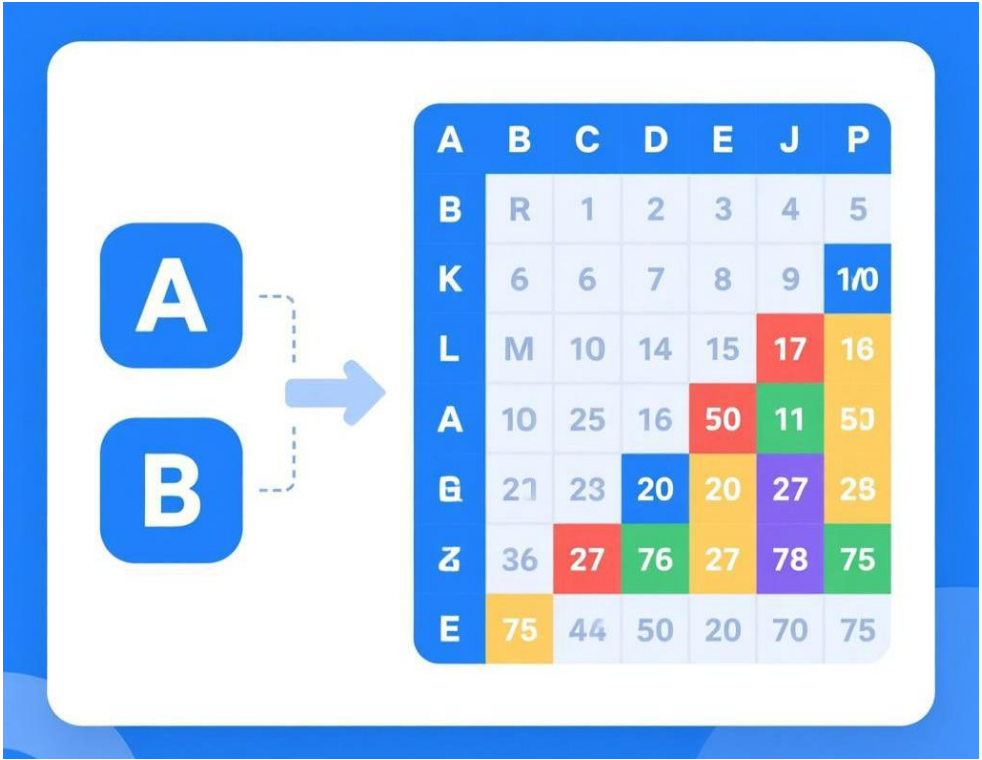
06 | LCS解回文串问题

07 | 总结





PART 01



问题与最优子结构



最长公共子序列

Longest Common Subsequence

问题定义

给定两个序列 X 和 Y ，找出它们共有的、**相对顺序不变**的最长子序列。子序列不要求连续，但必须保持原有顺序。

序列 X

A B C B D A B

序列 Y

B D C A B

最长公共子序列

B C A B (长度为 4)

LCS形式化定义

给定两个字符串：

$X_m = (x_1, x_2, \dots, x_m)$ 和 $Y_n = (y_1, y_2, \dots, y_n)$,

求一个最长的字符串序列 $Z_k = (z_1, z_2, \dots, z_k)$ 使得 Z 既是 X 的子序列也是 Y 的子序列，并且 k 尽可能大。这里的 k 即为 LCS 的长度。

暴力瓶颈

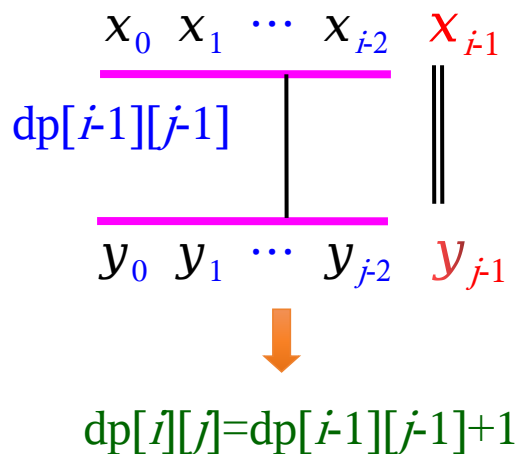
若序列长度分别为 m 和 n ，则子序列数量级为 $O(2^m)$ 和 $O(2^n)$ ，呈指数级增长，暴力枚举在时间和空间上都不可行。

一般来说动态规划算法设计要经历以下几个步骤

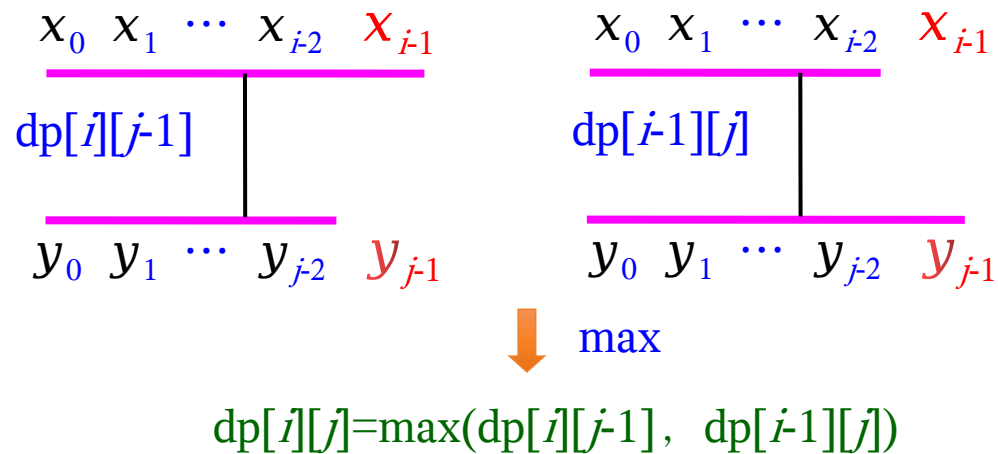
- ① **确定状态**：将问题求解中各个阶段所处的各种情况用不同的状态表示出来。
- ② **确定状态转移方程**：描述求解中各个阶段的状态转移和指标函数的关系。
- ③ **确定初始条件和边界情况**：状态转移方程通常是一个递推式，初始条件通常指定递推的起点，在递推中需要考虑一些特殊情况，称为边界情况。
- ④ **确定计算顺序**：也就是指定求状态转移方程的顺序，是顺序求解还是逆序求解。
- ⑤ **消除冗余**：如采用滚动数组进一步提高时空性能。

解

设计二维动态规划数组 dp ，其中 $dp[i][j]$ 为 $(x_0, x_1, \dots, x_{i-1})$ 和 $(y_0, y_1, \dots, y_{j-1})$ 的最长公共子序列长度，求 $dp[i][j]$ 的两种情况下。



(a) $x_{i-1} = y_{j-1}$



(b) $x_{i-1} \neq y_{j-1}$

状态转移方程

$dp[0][0]=0$

初始条件

$dp[i][0]=0$

边界情况

$dp[0][j]=0$

边界情况

$dp[i][j]=dp[i-1][j-1]+1$

$x[i-1]=y[j-1]$

$dp[i][j]=\max\{dp[i][j-1], dp[i-1][j]\}$

$x[i-1] \neq y[j-1]$

在求出dp数组后， $dp[m][n]$ 就是X和Y的最长公共子序列长度。

int dp[MAXN][MAXN];	//二维动态规划数组
int m,n;	//m和n分别为X,Y的长度
int LCSlength()	//求dp
{ dp[0][0]=0;	
for (int i=0;i<=m;i++)	//将dp[i][0]置为0,边界条件
dp[i][0]=0;	
for (int j=0;j<=n;j++)	//将dp[0][j]置为0,边界条件
dp[0][j]=0;	
for (int i=1;i<=m;i++)	
{ for (int j=1;j<=n;j++)	//两重for循环处理X、Y的所有字符
{ if (x[i-1]==y[j-1])	//情况①
dp[i][j]=dp[i-1][j-1]+1;	
else	//情况②
dp[i][j]=max(dp[i][j-1],dp[i-1][j]);	
}	
}	
return dp[m][n];	
}	

【算法分析】 算法的时间复杂度为 $O(mn)$ ，空间复杂度为 $O(mn)$ 。

```
int dp[MAXN][MAXN];
int m,n;
```

//二维动态规划数组
//m和n分别为X,b的长度

```
int LCSlength()
```

//求dp

```
{ dp[0][0]=0;
  for (int i=0;i<=m;i++)
    dp[i][0]=0;
  for (int j=0;j<=n;j++)
    dp[0][j]=0;
  for (int i=1;i<=m;i++)
  { for (int j=1;j<=n;j++)
    { if (x[i-1]==y[j-1])
        dp[i][j]=dp[i-1][j-1]+1;
      else
        dp[i][j]=max(dp[i][j-1],dp[i-1][j]);
    }
  }
  return dp[m][n];
}
```

//将dp[i][0]置为0,边界条件

//将dp[0][j]置为0,边界条件

//两重for循环处理X、Y的所有字符
//情况①

//情况②

例如，设所给的两个序列为
X=<A,B,C,B,D,A,B>
Y=<B,D,C,A,B,A>。

		j	0	1	2	3	4	5	6
i	x _i	y _j		B	D	C	A	B	A
0			0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖	←	↖
2	B		0	↖	←	←	↑	↖	←
3	C		0	↑	↑	↖	←	↑	↑
4	B		0	↖	↑	↑	↑	↖	←
5	D		0	↑	↖	↑	↑	↑	↑
6	A		0	↑	↑	↑	↖	↑	↖
7	B		0	↖	↑	↑	↑	↖	↑

当求出 dp 后如何利用 dp 求一个最长公共子序列呢？分析状态转移方程最后两行的计算过程可以看出：

- ① 若 $dp[i][j]=dp[i-1][j-1]+1$ ，则有 $x[i-1]=y[j-1]$ ，也就是说 $x[i-1]$ 或 $y[j-1]$ 是LCS中的字符。
- ② 若 $dp[i][j]=dp[i][j-1]$ ，则有 $x[i-1] \neq y[j-1]$ ，也就是说 $x[i-1]$ 或 $y[j-1]$ 不是LCS中的字符。
- ③ 若 $dp[i][j]=dp[i-1][j]$ ，则有 $x[i-1] \neq y[j-1]$ ，同样 $x[i-1]$ 或 $y[j-1]$ 不是LCS中的字符。

$dp[0][0]=0$

初始条件

$dp[i][0]=0$

边界情况

$dp[0][j]=0$

边界情况

$dp[i][j]=dp[i-1][j-1]+1$

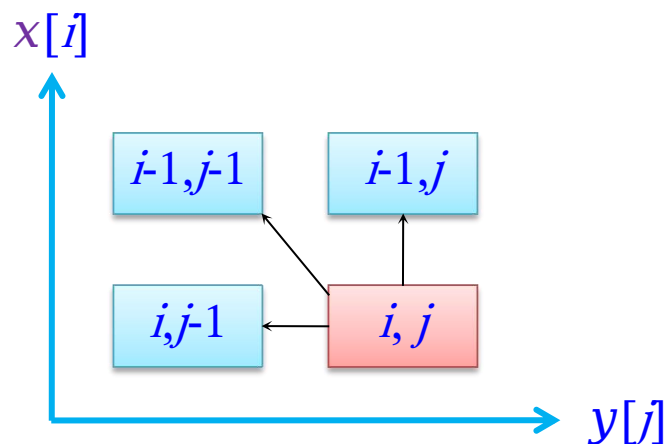
$x[i-1]=y[j-1]$

$dp[i][j]=\max\{dp[i][j-1], dp[i-1][j]\}$

$x[i-1] \neq y[j-1]$

用string容器subs存放一个LCS， $i=m$ ， $j=n$ 开始向subs中添加 $k=dp[m][n]$ 个字符，归纳为如下3种情况：

- ① 若 $dp[i][j]=dp[i-1][j]$ （即当前dp元素值等于上方相邻元素值），移到上一行即 i 减1，此时 $x[i]/y[j]$ 不是LCS字符。
- ② 若 $dp[i][j]=dp[i][j-1]$ （即当前dp元素值等于左边相邻元素值），移到左一列即 j 减1，此时 $x[i]/y[j]$ 不是LCS字符。
- ③ 其他情况只能是 $dp[i][j]=dp[i-1][j-1]+1$ ，移到左上方即 i 减1同时 j 减1，此时 $x[i]/y[j]$ 是LCS的字符，将 $x[i]/y[j]$ 添加到subs中。



例如, $X="abcbdb"$, $m=6$, $Y="acbbabdbb"$, $n=9$ 。

		b										
		a	c	b	b	a	b	d	b	b		
		a	0	1	2	3	4	5	6	7	8	9
a	0	0	0	0	0	0	0	0	0	0	0	0
b	1	0	1	1	1	1	1	1	1	1	1	1
c	2	0	1	1	2	2	2	2	2	2	2	2
b	3	0	1	2	2	2	2	2	2	2	2	2
d	4	0	1	2	3	3	3	3	3	3	3	3
b	5	0	1	2	3	3	3	3	4	4	4	4
	6	0	1	2	3	4	4	4	4	5	5	5

- ① 若 $dp[i][j]=dp[i-1][j]$, 移到上一行, 此时 $x[i]/y[j]$ 不是LCS字符。
- ② 若 $dp[i][j]=dp[i][j-1]$, 移到左一列, 此时 $x[i]/y[j]$ 不是LCS字符。
- ③ 若 $dp[i][j]=dp[i-1][j-1]+1$, 移到左上方, $x[i]/y[j]$ 是LCS的字符。

```
string getsubs()  
{  
    string subs="";  
    int k=dp[m][n];  
    int i=m,j=n;  
    while (k>0)  
    {  
        if (dp[i][j]==dp[i-1][j])  
            i--;  
        else if (dp[i][j]==dp[i][j-1])  
            j--;  
        else  
        {  
            subs+=x[i-1];  
            i--; j--; k--;  
        }  
    }  
    reverse(subs.begin(),subs.end());  
    return subs;  
}
```

//由dp构造subs
//存放一个LCS
//k为X和Y的最长公共子序列长度

//在subs中添加最长公共子序列(反向)

//subs中添加x[i-1]

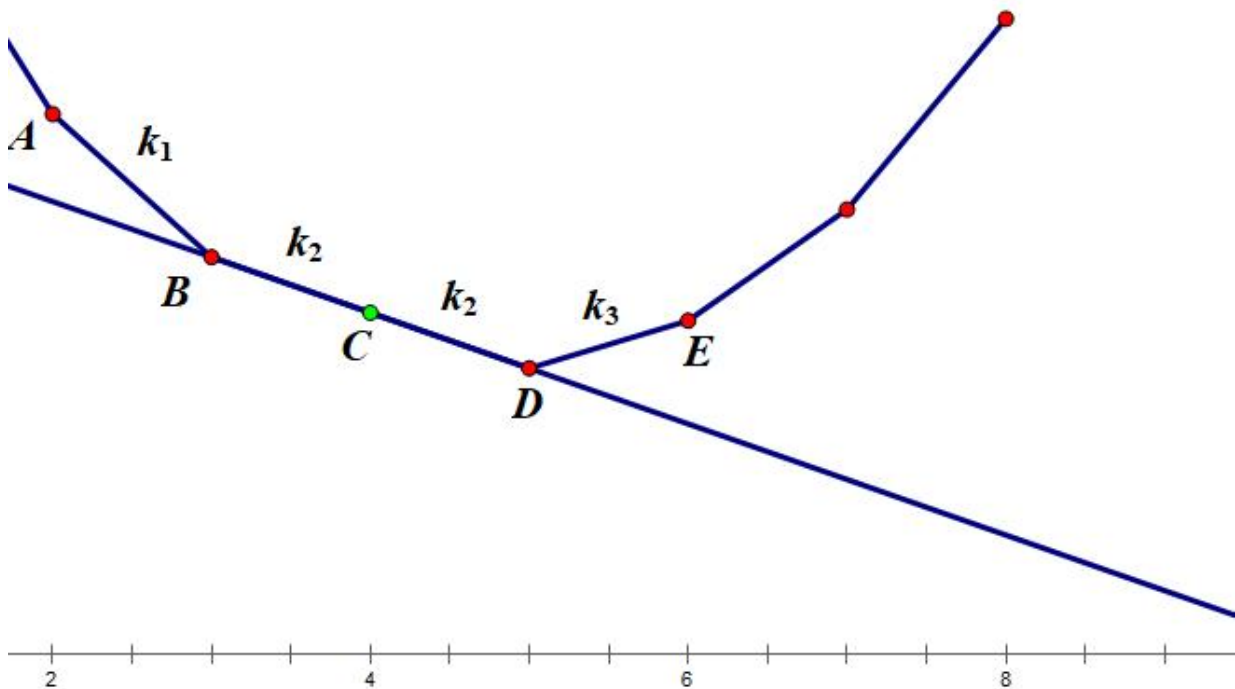
//逆置subs

LCS的空间优化

观察状态转移方程，我们发现计算 $dp[i][j]$ 时，只需要用到当前行 $dp[i]$ 和上一行 $dp[i-1]$ 的数据。

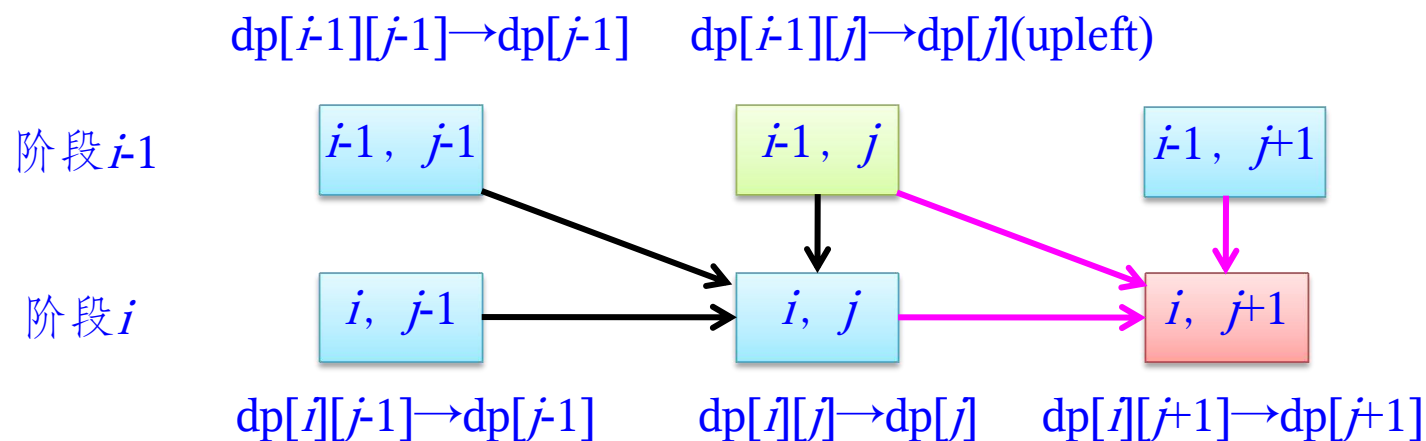
因此，我们可以将空间复杂度从 $O(mn)$ 优化到 $O(n)$ ，只用一个二维数组 $dp[2][n+1]$ 来交替存储两行数据。

注意：内层循环 j 必须正序遍历，因为 $dp[i][j]$ 可能依赖于同一行左边的值 $dp[i][j-1]$ 。



算法空间优化*

将 dp 改为一维数组，在阶段 $i-1$ （指考虑 $x[i-1]$ 字符的阶段）将 $dp[i-1][j-1]$ 存放在 $dp[j-1]$ 中，将 $dp[i-1][j]$ 存放在 $dp[j]$ 中，这两个状态是可以区分的。

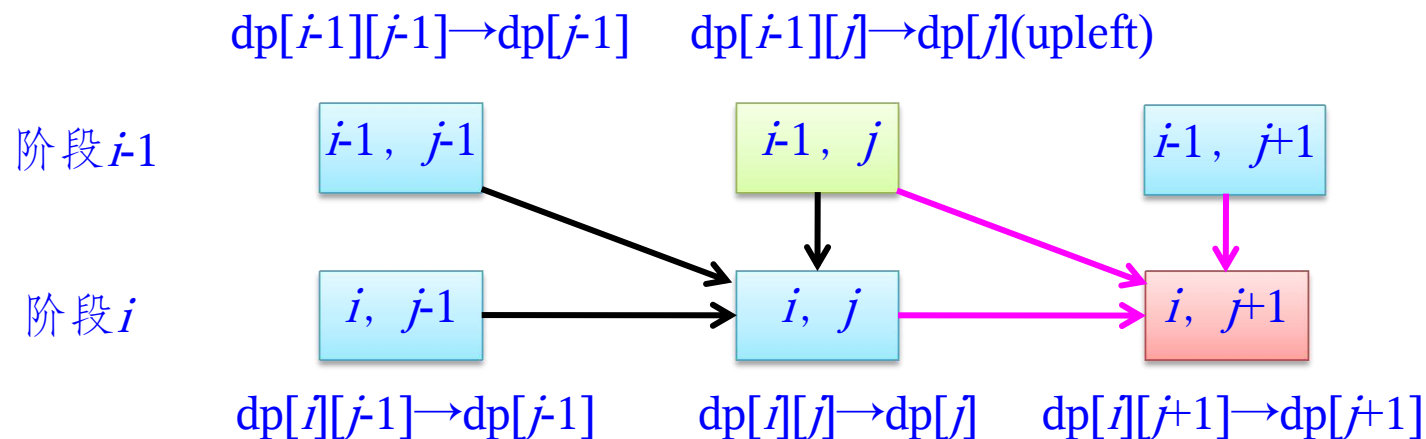


$$dp[i][j] = dp[i-1][j-1] + 1$$

$$dp[i][j] = \max\{dp[i][j-1], dp[i-1][j]\}$$

$$x[i-1] = y[j-1]$$

$$x[i-1] \neq y[j-1]$$



- 一个关键的问题是在阶段 i 中求 $dp[j+1]$ 时也与阶段 $i-1$ 的 $dp[j]$ 相关，此时已经在阶段 i 中修改了 $dp[j]$ ，为此用`upleft`变量记录 $dp[j]$ 修改之前的值`tmp`，以便在阶段 i 中求出 $dp[j]$ 后能够正确地求 $dp[j+1]$ 。
- 从中看出`upleft`是记录阶段 i 中每个位置的左上角元素，每个阶段 i 都是从 $j=1$ 开始的，所以初始置左上角元素`upleft=dp[0]`。

```
int LCSlength1()                                //求X和Y最长公共子序列长度的改进算法
{
    int m=a.length();                          //m为X的长度
    int n=b.length();                          //n为Y的长度
    vector<int> dp(n+1,0);                      //一维动态规划数组
    for (int i=1;i<=m;i++)
    {
        int upleft=dp[0];                      //阶段i初始化upleft
        for (int j=1;j<=n;j++)
        {
            int tmp=dp[j];                    //临时保存dp[j]
            if (a[i-1]==b[j-1])
                dp[j]=upleft+1;
            else
                dp[j] = max(dp[j-1],dp[j]);
            upleft=tmp;                        //修改upleft
        }
    }
    return dp[n];
}
```



PART 02



LCS解LIS技巧



最长递增子序列

问题描述：给定一个无序的整数序列 $a[0..n-1]$ ，求其中最长递增（严格）子序列的长度。

例如， $a=\{2, 1, 5, 3, 6, 4, 8, 9, 7\}$ ， $n=9$ ，其最长递增子序列为 $\{1, 3, 4, 8, 9\}$ ，结果为5。

技巧一：用LCS求解LIS

最长上升子序列 (LIS) 问题可以通过巧妙的映射转化为LCS问题。

1. 排序

对原数组 A 进行排序，得到新数组 B。

$A = [3, 1, 4, 1, 5]$

$B = [1, 1, 3, 4, 5]$



2. 求LCS

原数组 A 与排序数组 B 的LCS，即为 A 的LIS。

$LCS(A, B) = [1, 4, 5]$

该转换把LIS问题无缝嫁接到成熟的LCS框架，展示了算法组合的灵活性。

LCS解LIS: 复杂度与边界

复杂度分析



时间: $O(n^2)$ (排序 $O(n\log n)$ + LCS $O(n^2)$)



空间: $O(n^2)$ (DP表)

并非LIS最优解，但思路清晰，代码易复用。

适用场景



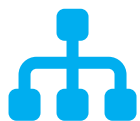
适用于需要同时求出具体序列，而不仅仅是长度的场景。



体现了以已知解未知的方法论价值，是算法思想迁移的典型范例。

解

- $dp[i]$ 表示以 $a[i]$ 结尾的子序列 $a[0..i]$ （共 $i+1$ 个元素）中的最长递增子序列的长度。
- 计算顺序是 i 从 0 到 $n-1$ ，对于每个 $a[i]$ ， $dp[i]$ 置为 1（表示只有 $a[i]$ 一个元素时最长递增子序列的长度为 1）。



最优子结构

问题可拆解为重叠子问题。



无后效性

未来决策不影响过去状态。



填表递推

自底向上计算并存储解。



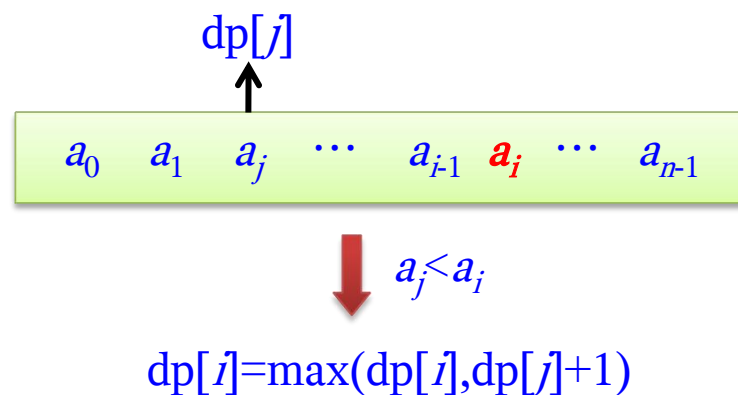
路径回溯

根据记录还原具体解。

掌握此范式可迁移至 **编辑距离、子串匹配** 等问题，形成解决序列类DP的通用思维模板。

考虑 $a[0..i-1]$ 中的每一个元素 $a[j]$ ，分为两种情况：

- ① 若 $a[j] < a[i]$ ，则以 a_j 结尾的最长递增子序列加上 a_i 可能构成一个更长的递增子序列，此时有 $dp[i] = \max(dp[i], dp[j] + 1)$ 。



- ② 否则最长递增子序列没有改变。

在求出 dp 数组后，通过顺序遍历 dp 求出最大值 ans ，则 ans 就是最长递增（严格）子序列的长度。

状态转移方程

$$\text{dp}[i]=1$$

$0 \leq i \leq n-1$ (初始条件)

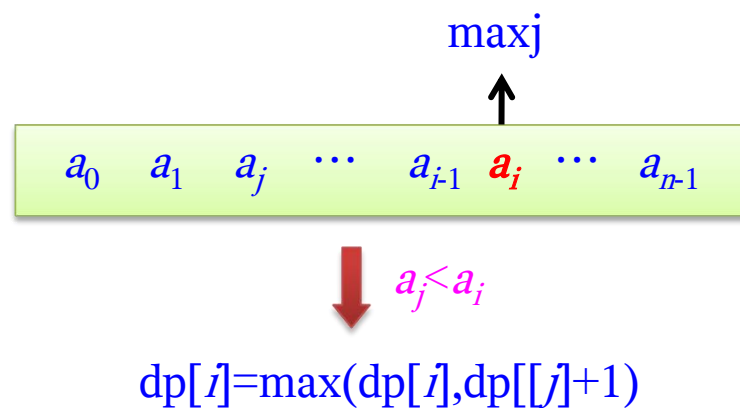
$$\text{dp}[i]=\max_{a[j]<a[i](j<i)}\{\text{dp}[j]+1\}$$

$0 \leq i \leq n-1$

```
int maxInclen(vector<int> &a)           //求最长递增子序列长度
{   int n=a.size();
    for(int i=0;i<n;i++)
    {   dp[i]=1;
        for(int j=0;j<i;j++)
        {   if (a[i]>a[j])
                dp[i]=max(dp[i],dp[j]+1);
            }
        }
    int ans=dp[0];
    for(int i=1;i<n;i++)
        ans=max(ans,dp[i]);
    return ans;
}
```

【算法分析】 时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

- 当求出dp后可以推导出一个最长递增子序列。
- 先在dp数组中求出最大元素的序号maxj，置j=maxj，prej从j的前一个序号开始在a中向前查找，rnum从dp[maxj]开始，若a[prej]<a[j]置rnum--，直到rnum为0，对应的a中子序列就是一个最大连续子序列。



```
vector<int> maxInc(vector<int>& a) //求一个最长递增子序列
{
    int n=a.size();
    vector<int> x;
    int maxj=0;
    for(int j=1;j<n;j++)
        if(dp[j]>dp[maxj]) maxj=j;
    x.push_back(a[maxj]);
    int rnum=dp[maxj]-1;           //剩余的元素个数
    int j=maxj;                   //j指向当前最长递增子序列的一个元素
    int prej=maxj-1;             //prej查找最长递增子序列的前一个元素
    while(prej>=0 && rnum!=0)
    {
        if(a[prej]<a[j])
        {
            rnum--;
            x.push_back(a[prej]);
            j=prej; prej--;
        }
        else prej--;
    }
    reverse(x.begin(),x.end());
    return x;
}
```



PART 03



LCS造最短回文串



利用最长公共子序列求解回文词的构造问题

回文词的构造问题描述

- 对于任意给定的一个字符串（由大写字母、小写字母和数字字符构成），都可以通过插入若干个字符的方法将其变成一个回文串。
- 例如：给定一个字符串Ab3bd，在插入两个字符后就可以变成一个回文串，如可以在A的左边添上一个d，再在最后一个d之前添上一个A，这样改造后的字符串为dAb3bAd，它是一个回文串。
- 那么对于任意给定的一个字符串，最少要插入几个字符才能将其变成回文串呢？

该问题是IOI2000中的一道经典题目，可以直接用动态规划方法来求解，但也可以巧妙地利用最长公共子序列问题来求解。

技巧：LCS造最短回文串

通过求原串与逆串的LCS，找到已对称的部分，从而确定最少补全字符。

原串 S

"AAB"



逆串 R

"BAA"

关键洞察

LCS(S, R) 即为 不需要修改 就能形成回文的部分。

最少补全字符数 = $\text{len}(S) - \text{len}(\text{LCS}(S, R))$

$\text{LCS}(\text{"AAB"}, \text{"BAA"}) = \text{"AA"} \text{ (长度为2)}$

构造最短回文串的步骤



1. 提取LCS

LCS作为回文的中心骨架。

LCS = "AA"



2. 补全单侧

对不在LCS中的字符，按原顺序在左侧插入。

左侧插入: "B"



3. 镜像对称

左半侧的镜像自动落在右半侧。

最终回文: "BAAB"

此过程再次证明DP不仅能计数，还能指导最优构造。



PART 07



总结与拓展



LCS算法核心回顾



最优子结构

问题可拆解为重叠子问题。



无后效性

未来决策不影响过去状态。



填表递推

自底向上计算并存储解。



路径回溯

根据记录还原具体解。

掌握此范式可迁移至 **编辑距离、子串匹配** 等问题，形成解决序列类DP的通用思维模板。

LCS应用图谱与开放思考

版本差异对比 (Diff)

通过比较文本行的LCS，找出文件间的相同与不同部分。

生物信息学

比对DNA或蛋白质序列，寻找相似性，推断功能或进化关系。

语音识别

将识别出的音素序列与标准词序列进行匹配，提高识别准确率。

核心思想

寻找 **最大公共部分**，以解决差异最小化或相似度最大化问题。



鼓励思考：如何将新问题映射为寻找“公共序列”？



THANKS!

实战—最长回文子串 (LeetCode5)



自学(无监督学习+强化学习)

